

Padrão Observer - Plataforma de Streaming com Notificações por Gênero

Este trabalho aplica o padrão de projeto comportamental Observer na simulação de uma plataforma de vídeo sob demanda, no estilo Netflix. A ideia central é simples: sempre que um filme ou série novo entra no catálogo, a plataforma deve avisar os usuários cadastrados, mas só aqueles que têm interesse no gênero daquele lançamento e que escolheram receber notificações.

O padrão Observer

O Observer é um dos padrões comportamentais do GoF e resolve um problema bem comum:

Como fazer um objeto avisar vários outros objetos quando algo muda nele, sem que ele precise conhecer os detalhes de quem está "ouvindo"? A solução é separar os papéis em duas interfaces: uma para quem é observado (o Subject) e outra para quem observa (o Observer).

O Subject mantém uma lista de Observers e, sempre que seu estado muda, percorre essa lista chamando um método padrão - no nosso caso, `update` - em cada um deles. Cada Observer decide, na sua própria implementação, o que fazer com a notificação recebida. Nesta simulação, quem é observado é a Plataforma (o catálogo de filmes e séries) e quem observa são os Usuários. A grande vantagem é que a Plataforma não precisa saber nada sobre as preferências de cada usuário, ela só dispara o aviso. Quem decide se aquilo interessa ou não é o próprio usuário, dentro do método `update`.

Estrutura do código e UML

O código ficou dividido em cinco classes/interfaces, dentro do pacote `padrao.observer`. A interface `Observer` define um único método, `update(tituloConteudo, genero)`; qualquer classe que queira "escutar" os lançamentos da plataforma precisa implementá-la. A interface `Subject` define como gerenciar os observadores, com os métodos `addObserver`, `removeObserver` e `notifyObservers`.

A classe `Plataforma` implementa `Subject`. Ela guarda uma lista de observadores e os dados do último conteúdo adicionado (título e gênero). Quando chamamos o método `adicionarConteudo`, ela atualiza esses dados e já chama `notifyObservers` automaticamente, assim ninguém esquece de disparar a notificação na hora de programar.

A classe `Usuario` implementa `Observer`. Cada usuário tem um nome, uma lista de gêneros de interesse (ação, comédia, terror, anime ou ficção científica) e um campo booleano que indica se ele quer ou não receber notificações. No método `update`, o usuário confere essas duas condições antes de exibir qualquer mensagem. Por fim, a classe `Main` monta o cenário de teste com cinco usuários e simula alguns lançamentos. Em termos de UML, dá para resumir assim: existem duas interfaces (`Subject` e `Observer`) e duas classes concretas (`Plataforma` e `Usuario`), cada uma implementando a sua respectiva interface. A `Plataforma`

se relaciona com Observer por meio de uma lista, uma associação um-para-muitos, já que pode ter vários usuários cadastrados ao mesmo tempo. Não existe uma relação direta entre Plataforma e Usuario - a Plataforma só enxerga "observadores genéricos", o que é justamente o ponto principal do padrão: baixo acoplamento entre quem avisa e quem é avisado.

O diagrama

Como o programa funciona na prática

Quando o programa roda, primeiro criamos um objeto Plataforma e cinco objetos Usuario, cada um com seu nome, sua lista de gêneros preferidos e indicando se está com as notificações ligadas ou desligadas. Em seguida, registramos todos na plataforma usando addObserver. A partir daí, toda vez que chamamos plataforma.adicionarConteudo("Nome do Filme", "Gênero"), o seguinte acontece nos bastidores: a plataforma guarda esse título e gênero como o último lançamento e chama notifyObservers(), que percorre a lista de usuários cadastrados chamando update(titulo, genero) em cada um. Dentro do update, cada usuário confere duas coisas: se as notificações estão ativadas para ele e se aquele gênero está na lista de interesses dele. Só quando as duas condições são verdadeiras é que a notificação aparece no console. Além disso, os usuários têm alguns métodos extras para deixar a simulação mais realista: inscreverGenero e cancelarInscricaoGenero, para mudar os interesses depois de já cadastrado, e ativarNotificacoes / desativarNotificacoes, para ligar e desligar o recebimento de avisos a qualquer momento. E claro, o removeObserver, que já vem da própria ideia do padrão, permite que um usuário cancele a conta e pare de receber qualquer notificação, mesmo que continue "interessado" nos gêneros.

Exemplo de execução com 5 usuários

Para testar, foram criados cinco usuários com perfis bem diferentes. A Ana gosta de ação e ficção científica e está com as notificações ativadas. O Bruno se interessa só por comédia, também com notificações ligadas. A Carla curte terror e anime. O Diego, apesar de estar inscrito em todos os gêneros, começa o programa com as notificações desligadas, ou seja, ele optou por não ser incomodado por enquanto. E por fim a Elisa, que acompanha apenas anime. O primeiro lançamento simulado é o filme "Duro de Matar 5", do gênero ação. Nesse momento, só a Ana recebe a notificação: ela é a única com interesse em ação e notificações ativas. O Diego também gosta de ação, mas como está com as notificações desligadas, não recebe nada - exatamente o comportamento que se queria simular, o de o usuário poder optar por não ser avisado. Depois desse primeiro aviso, simulamos uma mudança de comportamento: o Diego resolve ligar as notificações de novo, e o Bruno passa a se interessar também por terror, além de comédia. Em seguida chega o segundo lançamento, "Invocação do Mal 4", do gênero terror. Dessa vez, três pessoas recebem o aviso: a Carla, que já gostava de terror, o Bruno, que acabou de se inscrever em terror, e o Diego, que religou as notificações e também tem interesse em terror. Antes do terceiro lançamento, a Elisa decide cancelar a conta na plataforma, o que na prática a remove da lista de observadores. Quando chega "Attack on Titan: Temporada Final", do gênero anime, tanto a Carla quanto o Diego são notificados, pois os dois têm interesse em anime e notificações ativas, mas a Elisa não recebe nada - mesmo gostando de anime, ela não está mais cadastrada como observadora da plataforma. Esse exemplo mostra bem os três filtros

que o programa aplica antes de mandar uma notificação: primeiro, se o usuário ainda está cadastrado na lista de observadores; segundo, se ele tem as notificações gerais ativadas; e terceiro, se o gênero do lançamento está entre os interesses dele. Só passando pelos três é que a mensagem aparece no console.